



Appunti di Basi di Dati

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

1	DDL - Data Definition Language	4
1.1	Notazione	4
1.2	Schemi	4
1.2.1	Cosa sono?	4
1.2.2	Sintassi di creazione	4
1.2.3	Esempio: Creazione di uno schema vuoto	4
1.2.4	Esempio: Creazione di uno schema utilizzando tutte le opzioni disponibili	4
1.3	Tabelle	5
1.3.1	Cosa sono?	5
1.3.2	Sintassi di creazione	5
1.3.3	Esempio: Creazione di una tabella con elementi minimi	5
1.3.4	Esempio: Creazione di una tabella full optional	5
1.4	Definizione dei Dati: I Domini	6
1.4.1	Cosa sono?	6
1.4.2	Domini elementari	6
1.5	Domini definiti dagli utenti	6
1.5.1	Cosa sono?	6
1.5.2	Sintassi di creazione	6
1.5.3	Esempio: Creazione di dominio semplice	6
1.5.4	Esempio: Creazione di un dominio full optional	7
1.6	Vincoli intrarelazionale	7
1.6.1	Tipi di vincoli intrarelazionali	7
1.6.2	Esempio	7
1.7	Vincoli interrelazionali: Integrità referenziale	8
1.7.1	Cosa sono?	8
1.7.2	Sintassi	8
1.7.3	Esempio	8
1.8	Aggiornamento	9
1.8.1	Il problema dell'aggiornamento	9
1.8.2	Il problema delle violazioni	9
1.8.3	Reazioni a violazioni sulla tabella padre	9
1.8.4	Vincoli con nomi	10
1.9	Modifiche degli schemi: ALTER	10
1.9.1	A cosa serve?	10
1.9.2	Tipi di ALTER	10

1.9.3	Sintassi ALTER DOMAIN	10
1.9.4	Esempio di ALTER DOMAIN	10
1.9.5	Sintassi ALTER TABLE	11
1.9.6	Esempio di ALTER TABLE	11
1.10	Vincolo interrelazionale: UPDATE	11
1.10.1	A cosa serve	11
1.10.2	Esempi pratici	12
1.11	Vincolo interrelazionale: DROP	13
1.11.1	Cosa fa?	13
1.11.2	DROP DOMAIN	13
2	Operatore SELECT - Interrogazione di un database	15
2.1	A cosa serve?	15
2.2	Sintassi	15
2.3	Esempio	15
2.4	Istruzione AS	15
2.4.1	A cosa serve?	15
2.4.2	Sintassi	15
2.4.3	Esempio	15
2.5	Clausola FROM	16
2.5.1	Sintassi	16
2.5.2	Notazione punto	16
2.5.3	Esempio	16
2.6	Clausola WHERE	16
2.6.1	A cosa serve?	16
2.6.2	Operatori di confronto	16
2.6.3	Operatore LIKE	16
2.6.4	Operatore BETWEEN	17
2.6.5	Operatore IN	17
2.6.6	Predicati semplici	17
2.6.7	Precedenza di valutazione tra operatori booleani	17
2.6.8	Sintassi	17
2.6.9	Esempio generico	17
2.6.10	Esempio con LIKE	17
2.6.11	Esempio con BETWEEN	18
2.6.12	Esempio con IN	18
2.7	Clausola DISTINCT	18
2.7.1	A cosa serve?	18
2.7.2	Sintassi	18
2.7.3	Esempio	18

2.8	Espressioni	18
2.8.1	A cosa servono?	18
2.8.2	Funzioni predefinite	19
2.8.3	Funzioni per stringhe	19
2.8.4	Esempio di utilizzo di funzioni aritmetiche	19
2.8.5	Esempio di utilizzo di funzioni per stringhe	19
3	Operatore JOIN	20
3.1	A cosa serve?	20
3.1.1	Cosa è un predicato join?	20
3.2	Sintassi JOIN Legacy	20
3.2.1	Esempio	20
3.3	Sintassi JOIN SQL-2	20
3.3.1	Esempio	21
3.4	OUTER JOINING	21
3.4.1	Cosa fa?	21
3.4.2	LEFT OUTER JOIN (o LEFT JOIN)	21

1. DDL - Data Definition Language

1.1 Notazione

- **TERMINI DEL LINGUAGGIO** in maiuscolo
- <x> usato per isolare un termine x
- [x] indicano che il termine x è opzionale
- {x} indicano che il termine x può essere ripetuto da 0 a un numero arbitrario di volte
- | separa opzioni alternative

1.2 Schemi

1.2.1 Cosa sono?

Uno schema di base di dati è una collezione di oggetti: domini, tabelle, asserzioni, viste, privilegi, ...

1.2.2 Sintassi di creazione

```
CREATE SCHEMA
  [NomeSchema] -- Se omissso e' il nome dell'utente che ha lanciato il
               comando
  [[authorization] Autorizzazione] -- Termine del linguaggio
  {DefinizioneElementoSchema} -- Termine variabile
```

Dopo il comando **CREATE SCHEMA** compaiono le definizioni dei vari elementi.

Non è necessario che la definizione di tutti gli elementi avvenga contemporaneamente alla creazione dello schema, può avvenire in più fasi successive

1.2.3 Esempio: Creazione di uno schema vuoto

```
CREATE SCHEMA GestioneAzienda;
```

1.2.4 Esempio: Creazione di uno schema utilizzando tutte le opzioni disponibili

```
CREATE SCHEMA GestioneAzienda -- Nome dello schema
AUTHORIZATION utente_admin -- Autorizzazione
CREATE TABLE Impiegato ( -- Termine variabile
  id_impiegato INT PRIMARY KEY,
  nome VARCHAR(50),
```

```

    cognome VARCHAR(50)
)
CREATE VIEW VistaImpiegati AS -- Altro termine variabile
SELECT nome, cognome FROM Impiegato;

```

1.3 Tabelle

1.3.1 Cosa sono?

Una tabella è costituita da una collezione ordinata di attributi e da un insieme (eventualmente vuoto) di vincoli

1.3.2 Sintassi di creazione

```

CREATE TABLE NomeTabella(
    NomeAttributo DOMINIO [ValoreDiDefault][Vincoli]
    {NomeAttributo DOMINIO [ValoreDiDefault][Vincoli]}
    [AltriVincoli]
)

```

1.3.3 Esempio: Creazione di una tabella con elementi minimi

```

CREATE TABLE Studente (
    matricola INT PRIMARY KEY,
    nome VARCHAR(30)
);

```

1.3.4 Esempio: Creazione di una tabella full optional

```

CREATE TABLE Esame (
    codice_esame INT,
    nome_corso VARCHAR(50) NOT NULL,
    data_esame DATE DEFAULT CURRENT_DATE,
    voto INT CHECK (voto BETWEEN 18 AND 30),
    matricola_studente INT,

    -- Definizione della chiave primaria composta
    CONSTRAINT PK_Esime PRIMARY KEY (codice_esame, matricola_studente),

    -- Definizione della chiave esterna con vincolo di integrita
    referenziale
    CONSTRAINT FK_Studente FOREIGN KEY (matricola_studente)
        REFERENCES Studente(matricola)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

```

1.4 Definizione dei Dati: I Domini

1.4.1 Cosa sono?

I domini specificano i valori ammessi di ciascun attributo

1.4.2 Domini elementari

- **Testuali**

- **varchar(n)**: Stringa di lunghezza variabile tra 0 e n caratteri
- **char(n)**: Stringa di lunghezza pari ad n

- **Numerici**

- **Numerici Esatti**

- * **integer** / **smallint** : rappresentano valori interi
- * **numeric** / **decimal** : rappresentano i valori decimali

- **Numerici approssimati**

- * **real** / **double precision** : rappresentano valori a singola / doppia precisione in virgola mobile
- * **float** permette di richiedere la precisione che si desidera

- **boolean**: un booleano

- **blob, clob**: oggetti di grandi dimensioni

- costruiti da binari (blob)
- costruiti da caratteri (clob)

vengono trattati e memorizzati diversamente dagli altri, vengono usati per informazioni non strutturate o multimediali

1.5 Domini definiti dagli utenti

1.5.1 Cosa sono?

Sono dei domini definiti dall'utente partendo dai domini elementari

1.5.2 Sintassi di creazione

```
CREATE DOMAIN NomeDominio AS DominioElementare  
    [ValoreDefault]  
    [Constraints]
```

1.5.3 Esempio: Creazione di dominio semplice

```
CREATE DOMAIN Voto AS INT;
```

1.5.4 Esempio: Creazione di un dominio full optional

```
CREATE DOMAIN Voto AS SMALLINT
DEFAULT '0'
CHECK (value >= 18 AND value <= 30);
```

1.6 Vincoli intrarelazionale

Un vincolo intrarelazione è una regola che specifica delle condizioni all'interno di una relazione

1.6.1 Tipidi di vincoli intrarelazionali

Proprietà sempre valida all'interno di una relazione

- **NOT NULL:** Il valore deve essere non nullo
- **UNIQUE:** I valori devono essere non ripetuti
- **PRIMARY KEY:** Chiave primaria (una sola per tabella)
- **CHECK:** Definisce condizioni complesse (sia intra che inter relazionali)

1.6.2 Esempio

```
CREATE TABLE Dipendenti (
  -- 1. VINCOLO DI CHIAVE PRIMARIA (e di dominio NOT NULL implicito)
  ID_Dipendente INT PRIMARY KEY,

  -- 2. VINCOLI DI DOMINIO (Tipo dato, NOT NULL e lunghezza)
  Nome VARCHAR(50) NOT NULL,
  Cognome VARCHAR(50) NOT NULL,

  -- 3. VINCOLO DI CHIAVE UNICA (Unique)
  -- L'email deve essere unica per ogni dipendente, ma potrebbe essere
  -- temporaneamente assente (NULL)
  Email VARCHAR(100) UNIQUE,

  -- 4. VINCOLO DI DOMINIO con controllo sul range (CHECK)
  -- Lo stipendio deve essere un numero positivo
  Stipendio_Mensile DECIMAL(10, 2) CHECK (Stipendio_Mensile > 0),

  Data_Assunzione DATE NOT NULL,
  Data_Fine_Contratto DATE,

  -- 5. VINCOLO DI TUPLA (CHECK che coinvolge piu colonne)
  -- La data di fine contratto deve essere successiva alla data di
  -- assunzione
  CONSTRAINT CHK_Date_Contratto CHECK (Data_Fine_Contratto >
    Data_Assunzione)
);
```


1.7 Vincoli interrelazionali: Integrità referenziale

1.7.1 Cosa sono?

Esprimono un legame gerarchico (padre / figlio) fra tabelle.

Alcuni attributi della tabella figlio sono definiti **FOREIGN KEY** si devono riferire (**REFERENCES**) ad alcuni attributi della tabella padre che costituiscono una chiave.

I valori contenuti nella **FOREIGN KEY** devono essere sempre presenti nella tabella padre.

1.7.2 Sintassi

REFERENCES

Nella parte di definizione degli attributi con il costrutto sintattico **REFERENCES**

```
AttrFiglio CHAR(3) REFERENCES TabellaPadre(AttrPadre)
```

FOREIGN KEY

Oppure dopo la definizione degli attributi con i costruttori **FOREGIN KEY** e **REFERENCES**

```
FOREIGN KEY(AttrFiglio)
REFERENCES TabellaPadre(AttrPadre)
```

Quando si hanno più attributi da riferire, si utilizza sempre **FOREIGN KEY**

```
FOREIGN KEY(AttrFiglio1{, AttrFiglio2})
REFERENCES TabellaPadre(AttrPadre1{, AttrPadre2})
```

Se si omettono gli attributi destinazione, vengono assunti quelli della chiave primaria

```
AttrFiglio CHAR(3) REFERENCES TabellaPadre
```

1.7.3 Esempio

```
CREATE TABLE Infrazioni (
    Codice INTEGER PRIMARY KEY, -- Identificativo dell'infrazione (es.
    34321)
    Data DATE NOT NULL,
    Vigile INTEGER NOT NULL,
    Prov CHAR(2),
    Numero CHAR(6),

    -- Vincolo interrelazionale singolo (in linea o a livello tabella)
    FOREIGN KEY (Vigile) REFERENCES Vigili(Matricola),

    -- Vincolo interrelazionale composto (obbligatoriamente a fine
    tabella)
    FOREIGN KEY (Prov, Numero) REFERENCES Auto(Prov, Numero)
);
```

1.8 Aggiornamento

1.8.1 Il problema dell'aggiornamento

Studente				Esame			
Matr	Nome	Città	CDip	Matr	CodCorso	Data	Voto
34321	Luca	Mi	Inf	34321	1	25/02/01	28
53524	Giovanni	To	Mat	34321	2	14/12/01	30
64521	Emilio	Ge	Ing	64521	1	12/03/02	24
73321	Francesca	Vr	Mat	73321	4	18/01/02	18

Se elimino da Studente la tupla con matricola 34321, cosa succede alla tabella esame?

1.8.2 Il problema delle violazioni

Se viene eseguita una operazione di aggiornamento che porta alla violazione di un vincolo di integrità referenziale la base di dati diventa non valida.

Per evitare ciò, vengono definite un insieme di politiche per evitare che questo accada, permettendo di scegliere delle reazioni da adottare in caso di violazione.

Queste politiche vanno dichiarate nella dichiarazione DDL dello schema.

Per tutti gli altri tipi di vincoli invece a seguito di una violazione il comando di aggiornamento viene rifiutato segnalando l'errore.

Tipo di violazione

Inserimento o modifica di tupla che viola il vincolo sulla **tabella figlio** (interna)



Politica di reazione

In entrambi i casi il comando non viene eseguito

Inserimento o modifica di tupla che viola il vincolo sulla tabella padre (esterna)



Quattro politiche possibili

1.8.3 Reazioni a violazioni sulla tabella padre

- **CASCADE:** L'operazione di modifica / cancellazione viene propagata alla tabella intera
- **SET NULL:** NULL nella tabella figlio in entrambi i casi
- **SET DEFAULT:** Default nella tabella figlio in entrambi i casi
- **NO ACTION:** Rifiutata in entrambi i casi

1.8.4 Vincoli con nomi

A fini diagnostici è spesso utile sapere quale vincolo è stato violato a seguito di un'azione sul DB. A tale scopo è possibile associare dei nomi ai vincoli, ad esempio:

```
Stipendio INTEGER CONSTRAINT StipendioPositivo  
CHECK (Stipendio > 0)
```

```
CONSTRAINT ForeignKeySedi  
FOREIGN KEY (Sede) REFERENCES Sedi
```

1.9 Modifiche degli schemi: ALTER

1.9.1 A cosa serve?

Modifica oggetti

1.9.2 Tipi di ALTER

- **ALTER DOMAIN:** Modifica domini
- **ALTER TABLE:** Modifica tabelle

Quando si definisce un nuovo vincolo deve essere soddisfatto dalla istanza presente, altrimenti viene rifiutato

1.9.3 Sintassi ALTER DOMAIN

```
ALTER DOMAIN NomeDominio <  
    SET          DEFAULT ValoreDefault |  
    DROP        DEFAULT |  
    ADD         CONSTRAINT DefVincolo |  
    DROP        CONSTRAINT NomeVincolo >
```

1.9.4 Esempio di ALTER DOMAIN

```
-- 1. Definiamo un valore di default standard per tutti i nuovi  
    inserimenti  
ALTER DOMAIN VotoEsame  
    SET DEFAULT 18;  
-- 2. Aggiungiamo un vincolo per gestire anche l'estensione del voto con  
    la "Lode" (es. inserendo 31)  
ALTER DOMAIN VotoEsame  
    ADD CONSTRAINT ControlloloLode CHECK (VALUE >= 18 AND VALUE <= 31);  
-- 3. Rimuoviamo il valore di default inserito in precedenza se non è più  
    necessario  
ALTER DOMAIN VotoEsame  
    DROP DEFAULT;
```

```
-- 4. Rimuoviamo il vincolo sulla lode per tornare alle impostazioni base
ALTER DOMAIN VotoEsame
    DROP CONSTRAINT ControlloLode;
```

1.9.5 Sintassi ALTER TABLE

```
ALTER TABLE NomeTabella <
    ALTER COLUMN NomeAttributo <
        SET DEFAULT NuovoDefault |
        DROP DEFAULT > |
    DROP COLUMN NomeAttributo |
    ADD COLUMN DefAttributo |
    DROP CONSTRAINT NomeVincolo |
    ADD CONSTRAINT DefVincolo >
```

1.9.6 Esempio di ALTER TABLE

```
-- 1. Aggiungiamo una nuova colonna per memorizzare l'Email dello
    studente
ALTER TABLE Studente
    ADD COLUMN Email VARCHAR(100);
-- 2. Modifichiamo la colonna "Citta" esistente per impostare un valore
    di default predefinito
ALTER TABLE Studente
    ALTER COLUMN Citta SET DEFAULT 'Mi';
-- 3. Rimuoviamo la colonna "CDip" (Codice Dipartimento) se la tabella
    viene ristrutturata
ALTER TABLE Studente
    DROP COLUMN CDip;
-- 4. Aggiungiamo un vincolo di unicita (Unique) sulla colonna Email
    appena creata
ALTER TABLE Studente
    ADD CONSTRAINT EmailUnica UNIQUE (Email);
-- 5. Rimuoviamo un vecchio vincolo di controllo (se esistente)
    identificato dal suo nome
ALTER TABLE Studente
    DROP CONSTRAINT VerificaMatricola;
```

1.10 Vincolo interrelazionale: UPDATE

Quando si effettua una modifica (UPDATE) sui dati all'interno di una base di dati relazionale, è fondamentale garantire che i legami tra le tabelle rimangano consistenti. Il vincolo di integrità referenziale (Foreign Key) interviene in modo specifico per regolamentare cosa succede quando viene modificata una chiave.

1.10.1 A cosa serve

Il vincolo interrelazionale durante un'operazione di modifica ha lo scopo di impedire la creazione di "tuple orfane". Si considerano due scenari distinti:

- **Modifica sulla tabella figlio (interna):** Se si modifica il valore della chiave esterna nella tabella figlio, il database controlla semplicemente che il nuovo valore esista nella chiave primaria della tabella padre. Se non esiste, l'operazione viene bloccata.
- **Modifica sulla tabella padre (esterna):** Se si modifica il valore della chiave primaria nella tabella padre, tutti i record della tabella figlio che puntavano a quel vecchio valore rimarrebbero associati a una chiave non più esistente. Per evitare questa violazione, il DBMS applica delle specifiche *politiche di reazione*.

Sintassi dell'operazione

La definizione della politica di reazione per la modifica viene dichiarata in fase di creazione o alterazione della tabella figlio, subito dopo la specifica della chiave esterna (**FOREIGN KEY**).

La sintassi generica all'interno del linguaggio SQL DDL è la seguente:

```
FOREIGN KEY (AttributoTabellaFiglio) REFERENCES TabellaPadre(
    AttributoTabellaPadre) ON UPDATE [ RESTRICT | CASCADE | SET NULL | SET
    DEFAULT ]
```

Il significato delle quattro politiche possibili è il seguente:

1. **RESTRICT (o NO ACTION):** È il comportamento di default. La modifica sulla tabella padre viene rifiutata e solleva un errore se esistono tuple collegate nella tabella figlio.
2. **CASCADE:** La modifica viene propagata a cascata. Modificando il valore nella tabella padre, tutti i record corrispondenti nella tabella figlio verranno aggiornati automaticamente con il nuovo valore.
3. **SET NULL:** Il valore della chiave esterna nei record correlati della tabella figlio viene impostato a NULL (richiede che la colonna non abbia il vincolo NOT NULL).
4. **SET DEFAULT:** Il valore della chiave esterna nella tabella figlio viene reimpostato al valore di default precedentemente configurato per quella colonna.

1.10.2 Esempi pratici

Riprendiamo come riferimento le tabelle **Studente** (tabella padre, con chiave primaria **Matr**) ed **Esame** (tabella figlio, con chiave esterna **Matr** che fa riferimento a **Studente**).

Esempio 1: Politica CASCADE (Propagazione automatica)

Si supponga di voler modificare la matricola dello studente 'Luca' da 34321 a 99999. Configurando la tabella con la politica **CASCADE**:

```
ALTER TABLE Esame
    ADD CONSTRAINT fk_studente
    FOREIGN KEY (Matr) REFERENCES Studente(Matrx)
    ON UPDATE CASCADE;
```

Eseguendo l'istruzione `UPDATE Studente SET Matr = 99999 WHERE Matr = 34321;`, il database aggiornerà automaticamente anche tutte le righe all'interno della tabella **Esame** in cui la matricola era 34321, sostituendola con 99999.

Esempio 2: Politica RESTRICT (Blocco cautelativo)

Nel caso in cui le matricole debbano rimanere tassativamente immutabili per ragioni storiche e di sicurezza, si opta per RESTRICT:

```
ALTER TABLE Esame
  ADD CONSTRAINT fk_studente
  FOREIGN KEY (Matr) REFERENCES Studente (Matr)
  ON UPDATE RESTRICT;
```

Se si provasse ad eseguire la modifica della matricola 34321 nella tabella **Studente**, il DBMS bloccherebbe l'operazione restituendo un errore del tipo: *“Violazione del vincolo di chiave esterna: record correlati presenti nella tabella Esame”*.

Esempio 3: Politica SET NULL (Annullamento del legame)

Se l'esame sostenuto deve rimanere registrato nel sistema ma, in seguito alla modifica o riorganizzazione di una targa/codice identificativo padre, si decidesse temporaneamente di scollegarlo dallo studente:

```
ALTER TABLE Esame
  ADD CONSTRAINT fk_studente
  FOREIGN KEY (Matr) REFERENCES Studente (Matr)
  ON UPDATE SET NULL;
```

Cambiando la matricola nella tabella padre, le righe corrispondenti nella tabella **Esame** vedranno il proprio campo **Matr** trasformarsi in NULL, indicando che l'esame è al momento “orfano” di uno studente identificato.

1.11 Vincolo interrelazionale: DROP

1.11.1 Cosa fa?

Cancella oggetti dallo schema

Ha 2 opzioni:

- **RESTRICT:** Un oggetto non è rimosso se non è vuoto
- **CASCADE:** Viene rimosso l'oggetto e tutto ciò che è coinvolto nella definizione dell'oggetto

1.11.2 DROP DOMAIN

Cancellazione del dominio

Sintassi

```
DROP DOMAIN NomeDominio [ RESTRICT | CASCADE ];
```

Esempio di utilizzo

```
-- 1. Creazione del dominio
CREATE DOMAIN CodiceDipartimento AS CHAR(3)
    CHECK (VALUE IS NOT NULL);

-- 2. Utilizzo del dominio nella tabella "Studente"
CREATE TABLE Studente (
    Matr INT PRIMARY KEY,
    Nome VARCHAR(50),
    CDip CodiceDipartimento -- La colonna CDip usa il dominio appena
        creato
);

DROP DOMAIN CodiceDipartimento RESTRICT;
```

DROP TABLE

Cancellazione della tabella

Sintassi

```
DROP TABLE NomeTabella [ RESTRICT | CASCADE ];
```

Esempi

```
-- Creazione della tabella Padre
CREATE TABLE Studente (
    Matr INT PRIMARY KEY,
    Nome VARCHAR(50)
);

-- Creazione della tabella Figlio con il vincolo di Foreign Key
CREATE TABLE Esame (
    CodCorso INT,
    Matr INT,
    Voto INT,
    PRIMARY KEY (CodCorso, Matr),
    CONSTRAINT fk_studente FOREIGN KEY (Matr) REFERENCES Studente(Matrx)
);

DROP TABLE Studente RESTRICT;
-- Nota: Scrivere "DROP TABLE Studente;" produce lo stesso effetto
```

2. Operatore SELECT - Interrogazione di un database

2.1 A cosa serve?

È la istruzione utilizzata per effettuare interrogazioni al database

2.2 Sintassi

```
SELECT ListaAttributi  
FROM ListaTabelle  
[WHERE Condizione]
```

2.3 Esempio

```
SELECT Nome, Cognome, Stipendio  
FROM Impiegato  
WHERE Mansione = 'Ingegnere'
```

2.4 Istruzione AS

2.4.1 A cosa serve?

Viene utilizzato per fare ridenominazioni su attributi e anche su relazioni

2.4.2 Sintassi

```
SELECT AttrEspr [ [ AS ] Alias ] {, AttrEspr [ [ AS ] Alias ] }  
FROM Tabella [ [ AS ] Alias ] {, Tabella [ [ AS ] Alias ] }  
[ WHERE Condizione ]
```

2.4.3 Esempio

```
SELECT I.Nome, D.Citta AS citta_lavoro  
FROM Impiegato AS I, Dip AS D  
WHERE I.Dipart = D.Nome;
```


2.5 Clausola FROM

Specifica la/le tabelle/e su cui operare per determinare il risultato

2.5.1 Sintassi

```
SELECT ListaAttributi  
FROM Tabella1 [ [ AS ] Alias1 ] {, TabellaN [ [ AS ] AliasN ] }
```

2.5.2 Notazione punto

La clausola . permette di specificare a che relazione appartiene un attributo (o anche una tabella a un database)

2.5.3 Esempio

```
SELECT I.Nome , D.NomeDip  
FROM Impiegato AS I, Dip AS D  
WHERE I.Dipart = D.CodDip;
```

2.6 Clausola WHERE

2.6.1 A cosa serve?

È una espressione booleana che specifica una o più condizioni di selezione

2.6.2 Operatori di confronto

- =
- <>
- <= =>
- LIKE
- BETWEEN
- IS NULL
- IS NOT NULL

2.6.3 Operatore LIKE

L'operatore LIKE permette di esprimere dei "pattern" su stringhe di caratteri (regex) mediante la seguente notazione:

- _ indica un singolo carattere arbitrario
- % indica una stringa di caratteri arbitraria (anche lunga 0)

2.6.4 Operatore BETWEEN

L'operatore BETWEEN permette di esprimere condizioni di appartenenza a un intervallo

2.6.5 Operatore IN

L'operatore IN permette di esprimere condizioni di appartenenza a un insieme

2.6.6 Predicati semplici

Sono espressioni di valutazione su attributi mediante operatori di confronto

2.6.7 Precedenza di valutazione tra operatori booleani

1. NOT
2. AND
3. OR

2.6.8 Sintassi

```
WHERE [NOT] PredicatoSemplice {< AND | OR > [NOT] PredicatoSemplice}
```

2.6.9 Esempio generico

```
SELECT Nome, Cognome, Mansione
FROM Impiegato
WHERE ufficio = '10';
```

2.6.10 Esempio con LIKE

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Cognome LIKE '%i';
```

Figura 2.1: Restituire tutti i cognomi che finiscono con la lettera i

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Cognome LIKE 'R_ssi';
```

Figura 2.2: Restituirà "Rossi" (dove _ sostituisce la 'o'), ma non "Rossini" (perché _ impone un solo carattere)

2.6.11 Esempio con BETWEEN

```
SELECT Nome, Stipendio
FROM Impiegato
WHERE Stipendio BETWEEN 38 AND 60;
```

2.6.12 Esempio con IN

```
SELECT Cognome, ufficio
FROM Impiegato
WHERE ufficio IN ('10', '20');
```

2.7 Clausola DISTINCT

2.7.1 A cosa serve?

In SQL, le tabelle prodotte dalle interrogazioni possono contenere più righe identiche tra loro. I duplicati possono essere rimossi usando la parola chiave DISTINCT

2.7.2 Sintassi

```
SELECT DISTINCT ListaAttributi
FROM ListaTabelle
[ WHERE Condizione ]
```

2.7.3 Esempio

```
SELECT DISTINCT Cognome, Citta
FROM Impiegato
WHERE Stipendio > 40;
```

2.8 Espressioni

2.8.1 A cosa servono?

Una espressione usata nella clausola SELECT dà luogo ad una nuova "colonna" che non corrisponde a nessuna colonna della tabella su cui si effettua la selezione.

Questa nuova colonna è detta colonna virtuale.

Le colonne virtuali non sono fisicamente memorizzate, ma esistono solo come risultato delle interrogazioni.

Anche alle colonne virtuali è possibile assegnare un alias (con il costrutto AS)

2.8.2 Funzioni predefinite

SQL mette a disposizione alcune funzioni predefinite:

- **Stringhe:** UPPER, UCASE, LENGTH, ...
- **Date / Intervalli:** +, -, DATE, DAYOFWEEK, ...
- **Matematiche:** *, +, -, /, TAN, SQRT, SIN, ...
- **Informazioni di sistema:** USER, CURRENT_DATE, ...

2.8.3 Funzioni per stringhe

Gli operatori più comuni per le stringhe sono:

- **||:** Operatore di concatenazione
- **UPPER, UCASE:** Trasforma la stringa in caratteri maiuscoli
- **LOWER, LCASE:** Trasforma la stringa in caratteri minuscoli

2.8.4 Esempio di utilizzo di funzioni aritmetiche

```
SELECT Nome, Stipendio, Premioprod, (Stipendio + Premioprod) AS  
       stipendio_totale  
FROM Impiegato  
WHERE Mansione = 'Ingegnere';
```

2.8.5 Esempio di utilizzo di funzioni per stringhe

```
SELECT Nome, Cognome, citta, mansione  
FROM Impiegato  
WHERE UCASE(mansione) = 'INGEGNERE';
```

3. Operatore JOIN

3.1 A cosa serve?

L'operatore JOIN rappresenta un'importante funzionalità in quanto permette di correlare dati in tabelle diverse.

Matematicamente si tratta del prodotto cartesiano tra le 2 tabelle, che avrà un numero di righe pari al prodotto del numero di righe delle due tabelle.

Ad un operatore JOIN sono applicati uno o più predicati join.

3.1.1 Cosa è un predicato join?

Un predicato di join esprime una condizione che deve essere verificata dalle tuple del risultato dell'interrogazione.

Il predicato di join permette di estrarre dal DB un sottoinsieme opportuno del prodotto cartesiano.

3.2 Sintassi JOIN Legacy

Nella sintassi legacy, le tabelle da unire vengono elencate nella clausola FROM separate da una virgola, e la condizione di join viene specificata nella clausola WHERE

```
SELECT ListaAttributi
FROM Tabella1, Tabella2
WHERE Tabella1.Attributo = Tabella2.Attributo;
```

3.2.1 Esempio

```
SELECT Impiegato.Nome, Dip.Citta AS citta_lavoro
FROM Impiegato, Dip
WHERE Impiegato.Dipart = Dip.Nome;
```

3.3 Sintassi JOIN SQL-2

Lo standard SQL-2 ha introdotto una sintassi dedicata che separa chiaramente la logica di unione (la JOIN) dalle condizioni di filtro (la WHERE), rendendo le interrogazioni più leggibili e riducendo il rischio di errori nel dimenticare le clausole di collegamento

```
SELECT ListaAttributi
FROM Tabella1 [AS Alias1]
JOIN Tabella2 [AS Alias2] ON Tabella1.Attributo = Tabella2.Attributo;
```

3.3.1 Esempio

```
SELECT I.Nome, D.Citta AS citta_lavoro
FROM Impiegato AS I
JOIN Dip AS D ON I.Dipart = D.Nome;
```

3.4 OUTER JOINING

3.4.1 Cosa fa?

L'operatore OUTER JOIN è un'estensione della JOIN standard (Inner Join) utilizzata quando si desidera conservare nel risultato anche le tuple che non hanno corrispondenze nell'altra tabella.

Mentre l'Inner Join restituisce solo le righe che soddisfano la condizione di unione (ovvero quelle con corrispondenza in entrambe le tabelle), l'Outer Join garantisce che almeno una delle due tabelle (o entrambe) sia rappresentata completamente.

Esistono 3 tipologie di OUTER JOIN

3.4.2 LEFT OUTER JOIN (o LEFT JOIN)

Restituisce tutte le righe della tabella indicata a sinistra, per le righe della tabella che non hanno corrispondenza nella tabella di destra, vengono visualizzati valori NULL per tutti gli attributi proveniente dalla tabella di destra

Sintassi

```
SELECT colonne
FROM TabellaA A
LEFT JOIN TabellaB B ON A.chiave = B.chiave;
```